HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch

Alberto Dassatti, Bruno
Da Rocha Carvalho

Laboratoire de High Performance Coding

semestre printemps 2026

High Performance Coding (HPC) - Labo 2 Roofline

Temps à disposition : 4 périodes (2 séances)

Récupération du laboratoire dans l'archive sur Cyberlearn

1 Objectifs du laboratoire

Dans ce laboratoire, vous serez introduits au **modèle Roofline**, une méthode de visualisation permettant d'analyser et de comprendre les performances d'un programme sur une architecture donnée.

L'objectif principal n'est pas d'optimiser immédiatement un code, mais de comprendre:

- quels sont les **facteurs limitants** de la performance (calcul, mémoire),
- dans quels **domaines d'application** le modèle Roofline est pertinent,
- comment interpréter les performances observées à l'aide de mesures matérielles.

Vous serez guidés pas à pas à travers différents cas d'étude afin d'introduire progressivement les concepts nécessaires. L'objectif est qu'à l'issue de ce laboratoire, vous soyez capables de reproduire ce type d'analyse de manière autonome dans de futurs projets.

2 Cahier des charges

Dans ce laboratoire, deux approches complémentaires d'analyse de performance vous seront présentées:

1. une **approche manuelle**, basée sur l'analyse du code et des accès mémoire,
2. une **approche instrumentée**, exploitant l'outil de profilage matériel **LIKWID**.

Ces deux méthodes seront utilisées conjointement afin de:

- établir un lien entre le comportement théorique d'un algorithme et les mesures réelles,
- positionner un code sur un Roofline et interpréter son positionnement.

Le laboratoire s'appuie sur un code fourni, contenant plusieurs cas d'étude illustrant différents comportements

3 Outil de profilage : LIKWID

Dans ce laboratoire, vous utiliserez l'outil LIKWID. LIKWID est un outil open source de profilage et de monitoring matériel pour les processeurs multi-cœurs.

Il permet notamment de mesurer:

- le temps d'exécution,
- le trafic mémoire (caches et DRAM),
- le nombre d'instructions exécutées,
- l'énergie consommée,
- certaines opérations arithmétiques.
- et plein d'autres choses.

LIKWID fonctionne en exploitant les compteurs matériels du processeur (PMU).

4 Installation de LIKWID

- Sur Linux natif, vous pouvez installer le paquet `likwid` ou compiler depuis les sources.
- Consultez la documentation officielle: Github Likwid.
- Utilisez une version récente (par exemple `>= 5.3.0`).

Pour les architectures spécifiques, consultez la documentation afin de vérifier les groupes de performances disponibles.

Si toute fois vous avez un environnement non supporté, on vous invite à utiliser les machines de laboratoire et à installer likwid en suivant le lien de leur github.

5 Introduction au modèle Roofline

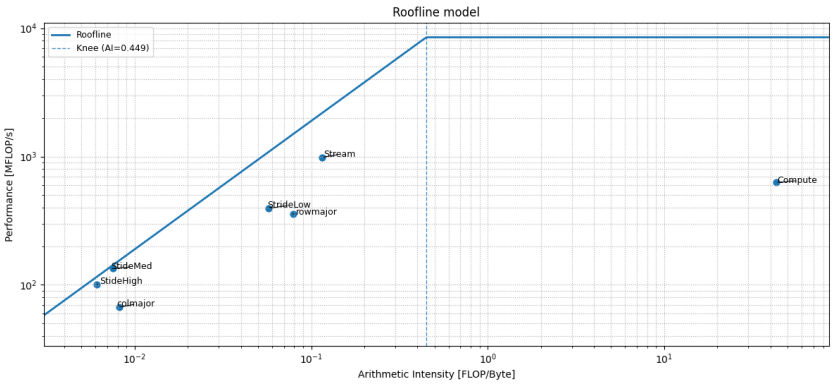
Le modèle Roofline permet de visualiser la performance maximale atteignable par un algorithme en fonction de son **intensité opérationnelle** (nombre d'opérations par octet transféré).

Il est défini par deux plafonds:

- la capacité de calcul maximale du processeur (`maxperf`),
- la bande passante mémoire maximale (`maxband`).

La performance théorique maximale est donnée par:

$$f(x) = \begin{cases} x \times \text{maxband} & \text{si } x \times \text{maxband} < \text{maxperf} \\ \text{maxperf} & \text{sinon} \end{cases}$$



L'axe des abscisses représente l'intensité arithmétique, également appelée intensité opérationnelle. Cette grandeur, notée **AI** (pour *Arithmetic Intensity*), est définie comme le rapport entre le nombre d'opérations effectuées et la quantité de données transférées depuis la mémoire. *Attention on parle vraiment d'opérations arithmètiques, non pas d'instructions en général.*

5.1 Construction du Roofline

5.1.1 Contexte

Dans notre contexte expérimental, les programmes étudiés s'exécutent en **séquentiel** (un seul thread). Il est donc essentiel de construire un Roofline **cohérent avec ce mode d'exécution**: les plafonds de performance (calcul et mémoire) doivent être mesurés **pour une exécution mono-thread**, et non pas à l'échelle de tous les cœurs du processeur.

Pourquoi ? Un Roofline représente le *maximum atteignable* dans les conditions d'exécution du programme. Comparer un code séquentiel au pic d'une machine entière (tous cœurs) conduirait à surestimer artificiellement la "sous-performance": le programme ne peut tout simplement pas exploiter des ressources qu'il n'utilise pas.

Pour construire le Roofline, vous devez déterminer expérimentalement:

- la performance maximale de calcul (`maxperf`),
- la bande passante mémoire maximale (`maxband`).

Ces deux valeurs se mesurent à l'aide d'outils fournis par `LIKWID` (aucun code à écrire). Dans notre cas, les mesures doivent être réalisées **dans la même configuration que l'application**, en particulier:

- **un seul thread**,
- la **même précision** que l'application (simple ou double précision),
- si possible, un benchmark adapté au **type d'opérations** (flottant ou entier).

5.1.2 Construction

La construction du modèle Roofline repose sur une bonne compréhension de l'architecture du processeur ainsi que sur la mesure expérimentale de ses plafonds de performance.

La première commande à utiliser est `likwid-topology`. Elle permet d'obtenir les informations essentielles sur l'architecture du processeur, notamment:

- le nom et le type du CPU,
- la topologie matérielle (nombre de sockets, de cœurs par socket et de threads par cœur),
- la hiérarchie et la taille des caches (L1, L2, L3).

Ces informations sont indispensables pour définir correctement le contexte expérimental. En particulier, elles permettent de déterminer le **nombre de threads réellement utilisables** et de raisonner sur les ressources matérielles disponibles par cœur, notamment la taille du cache L1.

Une fois ces informations collectées, il est possible de commencer la construction des bancs de tests à l'aide de la commande `likwid-bench`.

La performance maximale de calcul peut être estimée à l'aide des micro-benchmarks fournis par `LIKWID`. Néanmoins, une attention particulière doit être portée au choix du benchmark ainsi qu'aux paramètres d'exécution. En effet, il est essentiel de rester cohérent avec le contexte d'analyse afin d'éviter de *surestimer la sous-performance* de l'application étudiée.

Pour cela, il est nécessaire de:

- aligner le nombre de threads utilisés lors du benchmark avec celui du ou des programmes que l'on souhaite analyser ;

- garantir que la taille des données traitées par chaque thread n'introduise pas de biais lié à la bande passante mémoire, en s'assurant que ces données tiennent entièrement dans le cache L1.

Ce choix permet d'éliminer la mémoire comme facteur limitant et de mesurer un plafond de calcul reflétant uniquement les capacités des unités de calcul du processeur.

Exemple. Considérons un programme *mono-thread* effectuant uniquement des opérations en double précision. Le processeur utilisé dispose de 12 threads matériels et d'un cache L1 privé de 64 kB par cœur.

Dans ce contexte, le programme ne pouvant exploiter qu'un seul thread, le plafond de calcul doit être mesuré dans une configuration identique. Le benchmark est donc exécuté avec un seul thread, et la taille des données est choisie de manière à tenir entièrement dans le cache L1 du cœur utilisé. Afin de conserver une marge de sécurité et d'éviter toute interaction avec les niveaux de cache supérieurs, une taille de données de 30 kB est retenue.

Le benchmark peut alors être lancé de la manière suivante:

```
likwid-bench -t peakflops -W N:30kB:1
```

Cette configuration garantit que la mesure obtenue correspond au plafond de calcul effectivement atteignable par le programme, sans influence de la bande passante mémoire ni des ressources matérielles non exploitées.

La bande passante mémoire correspond au débit maximal de données pouvant être transférées entre la mémoire et les unités de calcul du processeur. Dans le modèle Roofline, ce plafond est généralement associé à la bande passante de la mémoire principale (DRAM), bien qu'il soit également possible de considérer d'autres chemins de données selon le contexte d'analyse.

La bande passante maximale peut être estimée théoriquement à partir des caractéristiques matérielles du processeur (nombre de sockets, de canaux mémoire, fréquence mémoire, etc.). Toutefois, dans la pratique, il est préférable de la mesurer expérimentalement à l'aide de benchmarks dédiés, tels que les micro-benchmarks mémoire fournis par `LIKWID`.

Comme pour le plafond de calcul, une attention particulière doit être portée au choix du benchmark mémoire. En effet, la bande passante n'est pas une valeur unique et immuable: elle dépend fortement du type d'accès mémoire. Sur les architectures modernes, les charges dominées par des lectures atteignent souvent un débit plus élevé que celles combinant lectures et écritures. Il est donc recommandé d'utiliser un benchmark dont le schéma d'accès mémoire se rapproche de celui de l'application étudiée, afin d'obtenir un plafond de bande passante aussi représentatif que possible.

Les micro-benchmarks `likwid-bench` proposent plusieurs noyaux permettant de mesurer le débit maximal de données, notamment:

- `load*`, correspondant à des accès en lecture seule ;
- `copy*`, impliquant une lecture et une écriture ;
- `stream*`, de la forme $A[i] = B[i] + \alpha \cdot C[i]$;
- `triad*`, de la forme $A[i] = B[i] + C[i] \cdot D[i]$.

Parmi ces noyaux, `load*` permet généralement d'atteindre la bande passante la plus élevée sur les processeurs x86, tandis que `stream*` et `triad*` offrent un comportement plus représentatif de nombreuses boucles applicatives.

Pour mesurer la bande passante mémoire maximale, la taille des données manipulées doit être suffisamment grande pour dépasser la capacité des caches et forcer des accès à la mémoire principale. De plus, l'option `-W` est utilisée afin d'assurer un placement mémoire correct des données (*first-touch*) pour chaque thread, garantissant ainsi un accès aussi local que possible aux ressources matérielles.

Exemple. Sur une machine disposant de 36 threads matériels, la bande passante mémoire maximale peut être mesurée à l'aide du noyau `load` avec un jeu de données largement supérieur à la taille du dernier niveau de cache. Le benchmark peut alors être exécuté comme suit:

```
likwid-bench -t load -W N:2GB:36
```

La valeur obtenue correspond au plafond de bande passante mémoire utilisé dans le modèle Roofline.

Les deux plafonds mesurés constituent les paramètres d'entrée du modèle Roofline et sont utilisés pour la génération du graphique final.

5.1.3 Graphing

Une fois les plafonds mesurés, les valeurs correspondantes sont inscrites dans un fichier `input.txt`, une valeur par ligne:

```
maxperf
maxband
```

Enfin, un script Python fourni génère automatiquement le graphique du Roofline à partir du fichier `input.txt`.

Le modèle Roofline ainsi construit fournit un cadre de référence cohérent pour l'analyse des performances des applications étudiées dans la suite de ce travail.

6 Construction d'un groupe de performance

Les groupes de performance fournis par `LIKWID` sont des configurations prédéfinies permettant de mesurer différentes métriques matérielles (FLOP/s, bande passante mémoire, énergie, etc.). Un groupe définit à la fois les **événements matériels** à observer (compteurs *core* et *uncore*) et les **métriques** calculées à partir de ces événements.

Cependant, ces groupes ne sont pas toujours disponibles sur toutes les architectures, ni nécessairement adaptés au contexte précis de ce laboratoire. Il est donc parfois nécessaire de **créer un groupe de performance personnalisé**.

Dans ce laboratoire, l'objectif n'est pas de mesurer la performance maximale théorique du processeur, mais de **positionner correctement un code simple dans le modèle Roofline**. Afin de limiter la complexité de l'analyse et de garantir la reproductibilité des résultats sur différentes infrastructures, le code étudié est compilé sans optimisation (`-O0`). Ce choix permet d'éviter toute vectorisation automatique et de se concentrer sur des **opérations scalaires en double précision**.

Dans ce contexte, le groupe de performance doit permettre de mesurer uniquement les quantités nécessaires à la construction et à l'exploitation du Roofline:

- le **temps d'exécution** de la région analysée ;
- le **nombre d'opérations arithmétiques scalaires en double précision** ;
- le **volume de données transférées depuis et vers la mémoire principale** (lectures et écritures DRAM).

Ces mesures sont suffisantes pour calculer:

- la performance en FLOP/s ;
- la bande passante mémoire effective ;
- l'intensité arithmetique (FLOP/Byte) du code analysé.

6.1 Sélection des événements matériels

Les événements matériels disponibles dépendent de l'architecture du processeur. Ils peuvent être listés à l'aide de la commande suivante:

```
likwid-perfctr -e
```

La sortie indique, pour chaque événement, son nom ainsi que le type de compteur utilisable (par exemple `PMC` pour les événements *core* ou `MBOX` pour les événements *uncore*). **Attention:** tous les événements ne peuvent pas être mesurés simultanément, le nombre de compteurs matériels étant limité.

Les compteurs de type `PMC` sont généralement utilisés pour mesurer des événements liés à l'exécution des instructions (opérations arithmétiques, instructions flottantes, etc.), tandis que les compteurs de type `MBOX` permettent de mesurer le trafic mémoire au niveau de la DRAM. Ces derniers sont souvent disponibles à l'échelle d'un **socket**, ce qui nécessite une interprétation prudente des métriques mémoire, en particulier lorsque plusieurs cœurs sont actifs.

6.2 Structure et déploiement d'un groupe personnalisé

Un groupe de performance LIKWID est décrit dans un fichier texte contenant:

- un champ **SHORT**, fournissant une description succincte du groupe ;
- un bloc **EVENTSET**, listant les événements matériels à mesurer ;
- un bloc **METRICS**, définissant les formules de calcul des métriques.

Une fois le groupe défini (par exemple dans un fichier `HPC.txt`), il peut être déployé dans le répertoire correspondant à l'architecture utilisée:

```
LIKWID_PATH=$(whereis likwid | awk '{print $2}')
cp HPC.txt "$LIKWID_PATH/perfgroups/{votre\_architecture}"
```

Le groupe peut ensuite être utilisé comme n'importe quel groupe fourni par LIKWID:

```
likwid-perfctr -C 2 -g HPC -m ./programme
```

Remarque importante (cohérence Roofline). Pour obtenir des métriques comparables à un Roofline *mono-thread*, les mesures doivent être réalisées dans une configuration cohérente avec l'application analysée: même nombre de threads, même précision numérique et interprétation correcte des compteurs mémoire (niveau cœur ou socket).

6.3 Liens et documentations utile

- Tous les groupes de performances de likwid
- Les descriptions des événements CPU de la machine de laboratoire, vous pouvez facilement depuis ce lien trouver votre machine.
- Example d'un groupe de performance qui mesure l'AI

7 Mesure par régions avec LIKWID

LIKWID permet de mesurer précisément des régions de code grâce à la **Marker API**. Cette approche permet de cibler directement, au sein du code, des régions spécifiques pour lesquelles on souhaite extraire des métriques de performance.

Pour cela, il est nécessaire d'inclure l'en-tête suivant:

```
#include <likwid-marker.h>
```

Puis d'entourer la région de code à analyser à l'aide des marqueurs:

```
LIKWID_MARKER_INIT;

LIKWID_MARKER_START("nom_region");
/* code à analyser */
LIKWID_MARKER_STOP("nom_region");

LIKWID_MARKER_CLOSE;
```

Lors de l'exécution, l'option `-m` est indispensable afin d'activer la collecte des mesures pour les régions marquées:

```
likwid-perfctr -C 2 -g <GROUPE> -m ./programme
```

Sans cette option, LIKWID mesurera l'ensemble du programme sans distinguer les différentes régions.

Choix du cœur d'exécution. L'option `-C` permet de fixer l'affinité du programme à un ou plusieurs cœurs matériels. Il est généralement recommandé d'éviter l'utilisation du cœur 0, car celui-ci est souvent sollicité par le système d'exploitation (interruptions, tâches de fond, gestion des périphériques), ce qui peut perturber les mesures de performance.

Le choix du ou des cœurs doit également être cohérent avec le contexte d'analyse:

- pour un programme séquentiel, un seul cœur matériel est suffisant ;
- pour un programme parallèle, le nombre de cœurs sélectionnés doit correspondre au nombre de threads effectivement utilisés.

Enfin, les identifiants passés à l'option `-C` correspondent aux **threads matériels** (hardware threads) tels qu'ils sont listés par la commande `likwid-topology`. Cette commande permet de visualiser la correspondance entre les identifiants, les cœurs physiques et les sockets, et constitue une référence indispensable pour un placement correct des threads.

8 Études de cas

Le code fourni contient plusieurs **cas d'étude**, chacun correspondant à une boucle simple mais présentant des comportements mémoire et calcul différents. Ces cas ont été conçus afin d'illustrer concrètement les principaux scénarios du modèle Roofline.

Pour chaque cas, deux types d'analyse complémentaires sont réalisées:

- une **analyse simplifiée "à la main"**, basée sur le code source, visant à estimer le nombre d'opérations et le volume de données manipulées ;
- une **analyse instrumentée**, reposant sur les mesures fournies par `LIKWID`.

L'analyse manuelle permet de raisonner sur le comportement algorithmique du code (nombre d'opérations par itération, accès mémoire, localité), et d'obtenir des *ordres de grandeur* pour l'AI et la bande passante. Elle est volontairement limitée et ne prend pas en compte de nombreux facteurs matériels tels que:

- la granularité des lignes de cache,
- les politiques de préchargement (*prefetching*),
- les écritures avec *write-allocate*,
- les effets de hiérarchie de caches,
- ou encore l'exécution hors ordre du processeur.

Ces limitations rendent difficile l'obtention de valeurs précises uniquement à partir d'un raisonnement théorique. En revanche, cette approche reste essentielle pour **comprendre et interpréter** les mesures fournies par les outils de profilage. Les valeurs issues de `LIKWID` constituent la référence pour le placement des points sur le Roofline, tandis que le calcul manuel sert de guide conceptuel.

Remarque sur le calcul manuel. Pour chaque cas d'étude, une analyse simplifiée du comportement algorithmique est également proposée directement dans le code source. Les fonctions correspondant au calcul manuel sont commentées afin d'expliciter le nombre d'opérations arithmétiques et les accès mémoire pris en compte par itération.

Ces commentaires visent à faciliter la compréhension des hypothèses utilisées pour estimer l'AI et la bande passante au niveau algorithmique. Ils ne constituent pas une mesure précise des performances, mais un support pédagogique permettant de relier le raisonnement théorique aux résultats obtenus à l'aide de `LIKWID`.

Pour chaque cas d'étude, vous devrez:

1. mesurer les performances à l'aide de `LIKWID` ;
2. placer le point correspondant sur le Roofline ;
3. interpréter le positionnement obtenu en lien avec le comportement mémoire et calcul observé.

8.1 Cas 1 : parcours séquentiel (HPC_STREAM)

Description: Lecture et écriture de tableaux en mémoire avec un parcours contigu.

Objectifs d'observation:

- trafic mémoire généré,

- bande passante atteinte,
- stabilité des mesures lorsque la taille augmente.

Commande recommandée:

```
likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_stream 50000000
```

8.2 Cas 2 : augmentation du calcul (HPC_COMPUTE)

Description: Même accès mémoire que le cas précédent, mais avec un nombre d'opérations arithmétiques plus élevé par élément.

Objectifs d'observation:

- effet de l'augmentation du calcul sans augmentation proportionnelle du trafic mémoire,
- évolution de la performance en fonction du paramètre `iters`.

Commandes recommandées:

```
likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_compute 20000000 200
```

8.3 Cas 3 : accès non contigu (HPC_STRIDE)

Description: Même opération que le cas HPC$_STREAM, maisavecunpas(stride)dansl'accsmmoire.$

Objectifs d'observation:

- impact de l'ordre d'accès mémoire,
- variation du temps et du trafic DRAM.

Commandes recommandées:

```
likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_stride 50000000 2
likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_stride 50000000 16
likwid-perfctr -C 2 -g HPC -m ./roofline_demo hpc_stride 50000000 64
```

8.4 Cas 4 : ordre de parcours 2D (ROW vs COL)

Description: Somme des éléments d'une matrice stockée en mémoire contiguë, avec deux ordres de boucles.

Objectifs d'observation:

- influence de la localité mémoire,
- comparaison directe de deux implémentations équivalentes.

Commandes recommandées:

```
likwid-perfctr -C 2 -g HPC -m ./roofline_demo rowmajor 4096
likwid-perfctr -C 2 -g HPC -m ./roofline_demo colmajor 4096
```

8.5 Travail demandé

Pour chaucun des 4 cas:

- Ajouter les points a vos roofline pour vous aider à analyser le comportement.
- Essayer de comprendre le code et ces problèmes avant de les lancer.
- Soyez curieux et testé plusieurs scénario et décriver en les impacts.
- Comparez les résultats entre eux.

Une fois que vous êtes bien imprégné de l'outil, n'hésitez pas à tenter de profiler votre code réalisé lors du lab01. Cela permettra déjà de chercher des pistes d'amélioration ou simplement de nommer les problèmes actuels de votre code.

9 Visualisation et interprétation

Ajoutez tous vos points dans le fichier `input.txt`:

```
maxperf
maxband
<AI> <Perf> hpc_stream
<AI> <Perf> hpc_stride64
<AI> <Perf> hpc_compute200
<AI> <Perf> rowmajor
<AI> <Perf> colmajor
```

Générez le graphique avec le script Python fourni.

10 Travail à rendre

Rédigez un rapport décrivant:

- votre Roofline,
- vos mesures LIKWID,
- vos analyses comparatives,
- vos propositions (théorique) d'amélioration du code.

Le rapport doit être concis et structuré.